

Decisiones de arquitectura del modelo que ayudaron a mejorar las métricas

Para el desarrollo del modelo se tomó como base una estrategia de *transfer learning* usando EfficientNet-B0, una arquitectura previamente entrenada con ImageNet. Esta fue una de las decisiones más importantes, porque permitió aprovechar el conocimiento que el modelo ya tenía sobre patrones visuales generales, en vez de entrenar toda la red desde cero.

Otra decisión relevante fue modificar el clasificador final de EfficientNet-B0. En lugar de usar una capa lineal sencilla, se construyó una cabeza de clasificación más completa, incluyendo Dropout, activación y ReLU.

La estructura final del clasificador quedó compuesta por:

- Dropout de 0.4.
- Capa lineal de 1280 a 512 neuronas.
- Activación ReLU.
- Batch Normalization.
- Dropout de 0.3.
- Capa lineal final hacia las 200 clases.

Fue clave la forma en que se entrenó el modelo. No se hizo todo el entrenamiento de una sola vez, sino en dos fases. En la primera fase se congeló el backbone de EfficientNet y solo se entrenó el clasificador final. Esta decisión permitió conservar las características visuales que el modelo ya había aprendido previamente y adaptar únicamente la parte encargada de clasificar las imágenes del nuevo problema.

Después, en una segunda fase, se descongeló el modelo completo para realizar un fine tuning más controlado. Sin embargo, no se usó la misma tasa de aprendizaje para todas las capas. Para el backbone se utilizó un *learning rate* más bajo, mientras que para el clasificador se mantuvo uno mayor.

La función de pérdida también se ajustó usando CrossEntropyLoss con *label smoothing* de 0.1. En vez de obligarlo a asignar toda la probabilidad a una sola clase, se suaviza un poco esa decisión. Esto ayuda a que el modelo aprenda de una manera menos rígida y mejore su desempeño en datos que no ha visto antes.

La elección de EfficientNet-B0 fue por el equilibrio que ofrece entre rendimiento y costo computacional. Aunque existen modelos más grandes, EfficientNet-B0 permite obtener buenos resultados sin requerir tantos recursos de memoria o

tiempo de entrenamiento. Esto lo convierte en una opción adecuada cuando se quiere mejorar el desempeño sin aumentar demasiado la complejidad del modelo.

Finalmente, se utilizó un scheduler CosineAnnealingLR para ajustar dinámicamente la tasa de aprendizaje durante el entrenamiento. En lugar de dejar un *learning rate* fijo, este scheduler lo va reduciendo de forma progresiva siguiendo una curva cosenoidal. Esto ayuda a que el modelo tenga un entrenamiento más estable y pueda llegar a una mejor solución al final del proceso.

En conclusión, la mejora de las métricas no se debió a una sola decisión, sino a la combinación de varias estrategias bien aplicadas. El uso de transferencia de aprendizaje, el ajuste del clasificador, el entrenamiento en dos fases, el *fine tuning* controlado, la regularización con Dropout y AdamW, el *label smoothing*, el aumento de datos y el scheduler de aprendizaje permitieron construir un modelo más robusto y con mejor capacidad de generalización para el dataset Tiny ImageNet.